

1. Roofline y Análisis de Bottleneck de Kernels NN

1.1. Contexto

Evaluación técnica de Vortex como núcleo candidato para el TFG derivado del de Chacón Alfaro (clúster RISC-V cooperativo para ML). Vortex ya implementa una jerarquía de caché parametrizable (I\$/D\$ privada, L2 por clúster, L3 global), con coherencia derivada por punto de coherencia (sin snooping), sobre núcleos **SIMT** (warps de hilos), no núcleos escalares independientes. La decisión tomada es aislar un núcleo SIMT de Vortex y reemplazar su caché por una L1+L2 propia (ver el anexo de implementación de la L1, rama `11.cache/definition_and_interfaces`).

1.2. Herramienta de medición

`ci/roofline_suite.py` ejecuta los cuatro benchmarks de kernels neuronales existentes en el repositorio de Vortex — `sgemm` (multiplicación de matrices), `conv3` (convolución 3×3), `softmax` y `relu` — bajo una configuración de hardware compartida, y grafica los cuatro puntos en un mismo diagrama roofline de FLOP/ciclo contra intensidad aritmética.

1.3. Línea base: desglose de stalls del pipeline

Configuración por defecto: 1 clúster, 1 núcleo, 4 warps, 4 hilos.

Cuadro 1: Línea base de rendimiento por kernel (warps=4)

Kernel	Tamaño	Idle scheduler	Stall scrb	IPC	Latencia load
matmul (sgemm)	-n128	58 %	44 %	0.417	21.7 ciclos
conv (conv3)	-n64	61 %	25 %	0.393	11.6 ciclos
softmax	-n128	65 %	8 %	0.354	23.3 ciclos
relu	-n65536	69 %	21 %	0.311	22.8 ciclos

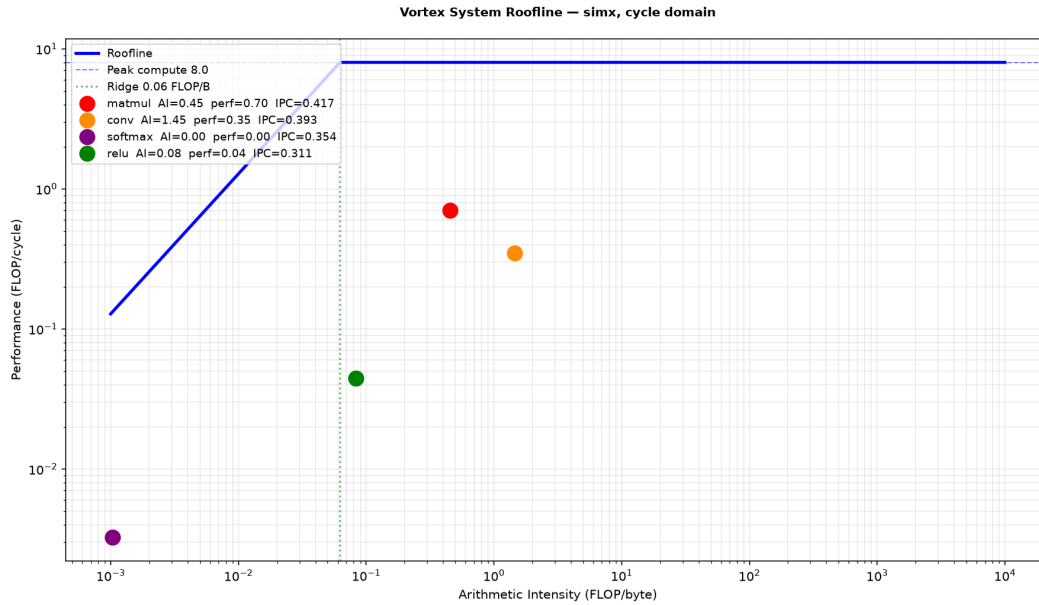


Figura 1: Roofline del sistema para los cuatro kernels NN, warps=4 (línea base).

Los cuatro puntos caen muy por debajo tanto del techo de cómputo como de la línea de ancho de banda de memoria — ninguno está limitado por FLOPs ni por ancho de banda. Los contadores de utilización de unidades funcionales (ALU, FPU, LSU, SFU) están prácticamente en 0 %. El limitador dominante es el *idle* del planificador (58–69 %): la mayoría de los ciclos no hay ningún warp listo para emitir, con el stall de *scoreboard* (*scrb*) como causa visible principal.

Causa raíz: ocupancia insuficiente para ocultar latencia. Con solo 4 warps residentes y latencias de *load* de 12 a 23 ciclos, en cuanto los 4 warps emiten una operación de latencia larga no queda ningún otro warp listo para ejecutar, y el núcleo permanece inactivo.

1.4. Verificación: sensibilidad a la cantidad de warps

Cuadro 2: Sensibilidad del IPC a la cantidad de warps

Kernel	IPC (warps=4)	IPC (warps=32)	Mejora
matmul	0.417	0.919	2.2×
conv	0.393	0.961	2.4×
softmax	0.354	0.740	2.1×
relu	0.311	0.993	3.2×

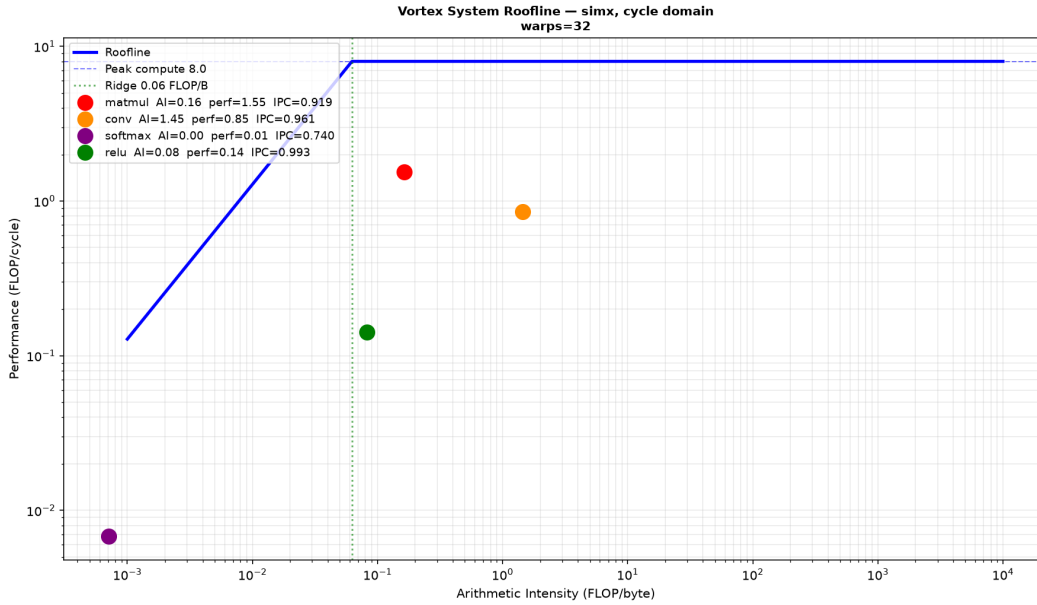


Figura 2: Roofline del sistema para los cuatro kernels NN, warps=32.

El kernel `relu` queda casi saturado (*idle*=1 %) — era puramente *latency-bound*. El kernel `softmax` mejora menos (*idle*=26 % incluso con 32 warps) porque su patrón de tres pasadas sobre los mismos datos genera más contención de MSHR y de banco (la latencia de *load* sube a 82 ciclos).

1.5. Aislar un núcleo

Configuración `tinygpu`:

```
CONFIGS=DVX_CFG_NUM_CLUSTERS=1 -DVX_CFG_NUM_CORES=1 \
-DVX_CFG_NUM_WARPS=2 -DVX_CFG_NUM_THREADS=2 \
-DVX_CFG_ICACHE_DISABLE -DVX_CFG_DCACHE_DISABLE -DVX_CFG_LMEM_DISABLE"
```

Núcleos usados en cada experimento

Todas las comparaciones de este anexo corren sobre **un solo núcleo** (default de Vortex, NUM_CLUSTERS=1, NUM_CORES=1, nunca cambiado):

Cuadro 3: Núcleos, warps e hilos usados por experimento

Experimento	Núcleos	Warps	Hilos	Caché
Línea base (Sec. 1.3)	1	4	4	con caché
Sensibilidad a warps	1	32	4	con caché
Núcleo aislado, con caché	1	2	2	con caché
Núcleo aislado, sin caché (tinygpu)	1	2	2	sin caché

Resultado medido: con caché vs. sin caché

Cuadro 4: IPC con caché vs. sin caché, mismo núcleo aislado

Kernel	IPC con caché	IPC sin caché	Caída	ifetch_lat con	ifetch_lat sin
matmul	0.214	0.076	2.8×	5.0 ciclos	22.0 ciclos
conv	0.201	0.077	2.6×	5.0 ciclos	21.1 ciclos
softmax	0.180	0.073	2.5×	5.0 ciclos	21.2 ciclos
relu	0.169	0.075	2.3×	5.0 ciclos	20.5 ciclos

Quitar la caché cuesta 2.3–2.8× de IPC, y el causante principal es el **fetch de instrucciones** (5 → ~21 ciclos sin I\$).

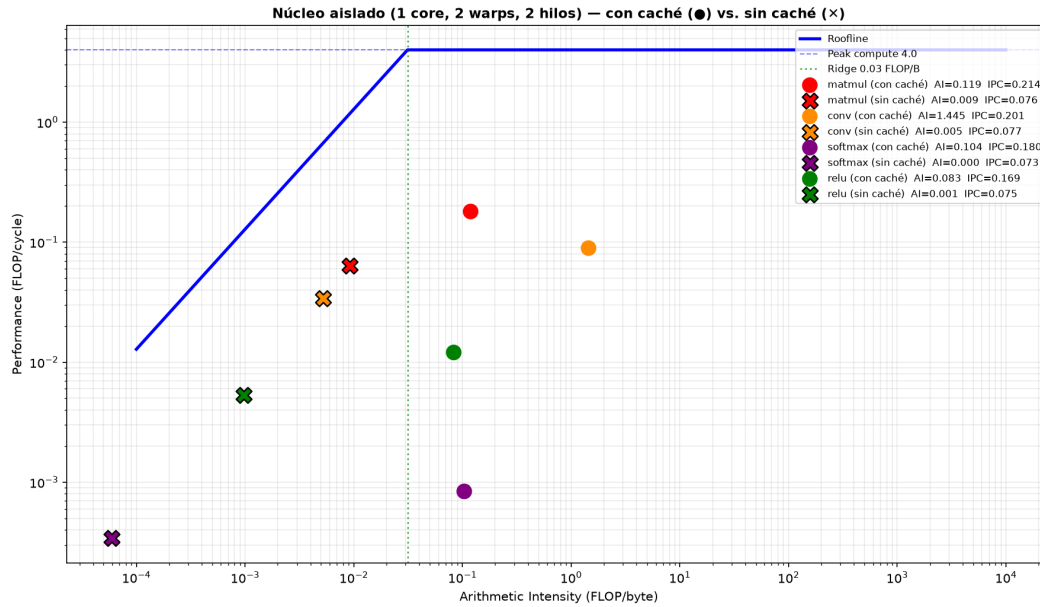


Figura 3: Roofline del núcleo aislado, con caché (●) vs. sin caché (×).

Sin caché, además de rendir menos, la AI también cae 1–2 órdenes de magnitud (ej. matmul: 0.119 → 0.009 FLOP/byte).

Formato del bus en el punto de aislamiento

`VX_mem_bus_if` es un canal valid/ready simple. La divergencia SIMT ya está resuelta antes de llegar acá: un `VX_mem_coalescer` funde las peticiones por-hilo del LSU en peticiones más anchas antes de que salgan del núcleo. Una L1 de reemplazo recibe una petición coalescida por operación de memoria, no accesos divergentes por hilo.

1.6. Relación con los objetivos del proyecto

Trabajo realizado	Objetivo / actividad del anteproyecto
<code>roofline_suite.py</code> + desglose de stalls	Instrumentación pedida en OE1 / A-2 / A-3.
Tabla 1	Línea base (A-2) contra la que comparar la L1+L2 nueva en A-22/A-23.
Hallazgo de latencia/ocupancia como limitador dominante	Insight de diseño para A-3.
Configuración <code>tinygpu</code>	Punto de partida técnico de A-1.
Formato de <code>VX_mem_bus_if</code> + <code>coalescer</code>	Resuelve el primer pendiente de A-4.

Cuadro 5: Mapeo del trabajo de evaluación a los objetivos del anteproyecto

1.7. Pendientes

- Definir cómo mapear coherencia MESI sobre un solo núcleo aislado.
- Separar cuánto de la caída con/sin caché es I\$ vs. D\$.
- Experimento con más de un núcleo.